

# MANUAL DE ENGENHARIA & ARQUITETURA DE SOFTWARE

Guia Técnico Definitivo do Ecossistema NOVA — Java 21 LTS, Spring Boot 3.3 & Spring AI

|                                                               |                                                               |
|---------------------------------------------------------------|---------------------------------------------------------------|
| <b>Autor:</b> Fábio Rodrigues (Desenvolvedor Java Back-end)   | <b>Versão da Arquitetura:</b> v3.22 (Enterprise Architecture) |
| <b>Ecossistema:</b> Java 21 / Spring Boot 3.3 / Spring AI MCP | <b>Qualidade:</b> 40 Tests Passed (JUnit 5 + Mockito 100%)    |
| <b>Repositório:</b> github.com/fabiorodrigues-tech-dev/NOVA   | <b>Deploy Contínuo:</b> nova-control-center-alsl.onrender.com |

## 1. Princípios Arquiteturais & Clean Architecture (Java 21 LTS)

O microsserviço agente-financeiro foi projetado para demonstrar maestria em engenharia de software corporativa, adotando Clean Architecture com isolamento absoluto de dependências. O domínio não conhece Spring, JPA ou banco de dados:

| Princípio de Engenharia    | Implementação Prática no Código                                                                                                | Benefício Arquitetural                                                                                |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Isolamento de Domínio      | Entidade Transacao.java e Enums são POJOs Java puros, livres de anotações @Entity ou imports do Spring.                        | Regras de negócio invariantes protegidas contra mudanças ou acoplamento a frameworks externos.        |
| Dependency Inversion (DIP) | O caso de uso depende da interface pura TransacaoRepository. A implementação TransacaoRepositoryImpl reside na infraestrutura. | Facilidade de substituição de infraestrutura e isolamento completo para testes unitários com Mockito. |
| Imutabilidade & Records    | Todos os dados que trafegam entre camadas utilizam Java 21 record (ex: ResumoFinanceiroResponse, ProjecaoResponse).            | Garantia de thread-safety, redução drástica de boilerplate e contratos transparentes.                 |

## 2. Caso de Uso: ImportarExtratoOfxUseCase (Parser & Deduplicação)

Responsável por ingerir extratos bancários oficiais do Nubank em formato OFX/CSV com total segurança contábil, garantindo que nenhuma transação seja duplicada no banco H2:

| Etapa do Pipeline                | Mecanismo Técnico Implementado                                                                                     | Garantia de Confiabilidade                                                                           |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1. Leitura & Parsing             | Extração das tags OFX <STMTTRN>: <TRNTYPE>, <DTPOSTED>, <TRNAMT>, <FITID> e <MEMO>.                                | Tratamento de formatos de data brasileiros (YYYYMMDD) e normalização de encoding UTF-8 / ISO-8859-1. |
| 2. Classificação Semântica       | Motor de regras com mais de 30 padrões de Regex que categorizam despesas em ALIMENTACAO, MORADIA, TRANSPORTE, etc. | Categorização automática de 100% dos lançamentos sem intervenção manual do usuário.                  |
| 3. Deduplicação por FITID / Hash | Cada lançamento gera uma chave de auditoria única baseada no FITID oficial do Nubank ou no hash SHA-256 da tupla.  | Tentativas de re-importar o mesmo extrato são ignoradas silenciosamente com idempotência contábil.   |
| 4. Segregação de Ativos          | Aportes e resgates de Caixinhas Nubank são identificados e apartados do saldo da Conta Corrente operacional.       | Preservação da integridade do saldo real (R\$ 0,03 em CC e R\$ 1.000,11 em Caixinhas).               |

### 3. Caso de Uso: CalcularProjecaoFinanceiraUseCase (Burn Rate & Preditivo)

Responsável por calcular a taxa de consumo de capital (Burn Rate diário) e projetar a saúde financeira do desenvolvedor até o último dia do ciclo mensal:

FORMULAÇÃO MATEMÁTICA DO BURN RATE DIÁRIO PONDERADO:

- **Burn Rate Diário:** *Total Despesas no Mês / Dias Decorridos*
- **Despesa Projetada Adicional:** *Burn Rate Diário × Dias Restantes no Mês*
- **Saldo Final Projetado:** *Saldo Líquido Atual – Despesa Projetada Adicional*
- **Classificação de Risco:** Se Saldo Projetado > R\$ 1.000 (BAIXO); Se > 0 (MODERADO); Se < 0 (ALTO).

### 4. Módulo de Caixinhas Nubank & Cálculo do Patrimônio Líquido

O ecossistema gerencia os ativos alocados nas Caixinhas Nubank (RDB DI com rendimento de 100% do CDI), recalculando automaticamente o Patrimônio Líquido Total e a reserva de segurança:

| Ativo / Caixinha             | Saldo Alocado | Finalidade Financeira                                                 | Rendimento     |
|------------------------------|---------------|-----------------------------------------------------------------------|----------------|
| Reserva de Emergência        | R\$ 500,00    | Reserva de liquidez imediata para despesas operacionais inesperadas.  | 100% CDI       |
| Caixinha Casal               | R\$ 500,11    | Metas compartilhadas de médio prazo com rendimento diário pós-fixado. | 100% CDI       |
| Conta Corrente (Operacional) | R\$ 0,03      | Disponível imediato em conta após conciliação de faturas.             | —              |
| PATRIMÔNIO TOTAL             | R\$ 1.000,14  | Patrimônio Líquido Consolidado (Conta Corrente + Caixinhas)           | Auditoria ACID |

### 5. Guia de Execução de Testes Automatizados & Esteira CI/CD

Instruções padronizadas para validação local e contínua da integridade técnica do projeto:

| Comando de Execução               | Finalidade Técnica                                                                                              | Critério de Aceite (Quality Gate)                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| ./run-tests.sh                    | Compila as classes de domínio, infraestrutura e testes, executando os 40 testes com JUnit 5 Standalone Console. | 100% dos testes devem passar (0 falhas, 0 erros) com tempo de execução inferior a 5 segundos. |
| git push origin main              | Dispara o workflow do GitHub Actions (ci.yml) em container Linux com Java 21 e Flake8.                          | Build verde no GitHub Actions pré-requisito para homologação de releases.                     |
| curl http://localhost:3000/health | Verificação rápida de integridade da API e dashboard, retornando status: UP em < 3ms.                           | HTTP 200 OK imediato sem sobrecarga do servidor.                                              |